

CS 2420: Algorithms and Data Structures

Homework 3 - SOLUTION KEY

Professor Smith

Due: October 15, 2024

Problem 1: Big O Analysis (25 points)

Question: Analyze the time complexity of the following code snippets and provide the Big O notation. Justify your answer.

Part A (5 points)

```
1 def mystery_function_a(n):
2     total = 0
3     for i in range(n):
4         for j in range(n):
5             total += i * j
6     return total
```

Solution: The outer loop runs n times, and for each iteration of the outer loop, the inner loop runs n times. The operations inside the nested loops (multiplication and addition) are constant time $O(1)$. Therefore, the total time complexity is $O(n \cdot n \cdot 1) = O(n^2)$.

Part B (10 points)

```
1 def mystery_function_b(n):
2     if n <= 1:
3         return 1
4     else:
5         return mystery_function_b(n//2) + mystery_function_b(n//2)
```

Solution: This is a recursive function where each call makes two recursive calls with input size $n/2$. Using the Master Theorem with $a = 2$, $b = 2$, and $f(n) = O(1)$: $\log_2 2 = 1$ - $f(n) = O(1) = O(n^0)$

Since $f(n) = O(n^0)$ and $0 < 1$, we're in case 1 of the Master Theorem. Therefore, $T(n) = O(n^{\log_2 2}) = O(n)$.

Part C (10 points)

```
1 def mystery_function_c(arr):
2     n = len(arr)
3     for i in range(n):
4         for j in range(i, n):
5             print(arr[i] + arr[j])
```

Solution: The outer loop runs n times. For each value of i , the inner loop runs $(n - i)$ times: - When $i = 0$: inner loop runs n times - When $i = 1$: inner loop runs $(n - 1)$ times - ... - When $i = n - 1$: inner loop runs 1 time

Total iterations: $n + (n - 1) + (n - 2) + \dots + 1 = \frac{n(n+1)}{2} = O(n^2)$

Problem 2: Sorting Algorithms (30 points)

Question: Consider the array [64, 34, 25, 12, 22, 11, 90].

Part A (15 points)

Show the step-by-step execution of Bubble Sort on this array.

Solution: Initial: [64, 34, 25, 12, 22, 11, 90]

Pass 1: - Compare 64,34 → swap: [34, 64, 25, 12, 22, 11, 90] - Compare 64,25 → swap: [34, 25, 64, 12, 22, 11, 90] - Compare 64,12 → swap: [34, 25, 12, 64, 22, 11, 90] - Compare 64,22 → swap: [34, 25, 12, 22, 64, 11, 90] - Compare 64,11 → swap: [34, 25, 12, 22, 11, 64, 90] - Compare 64,90 → no swap: [34, 25, 12, 22, 11, 64, 90]

Pass 2: [25, 34, 12, 22, 11, 64, 90] Pass 3: [25, 12, 22, 11, 34, 64, 90] Pass 4: [12, 22, 11, 25, 34, 64, 90] Pass 5: [12, 11, 22, 25, 34, 64, 90] Pass 6: [11, 12, 22, 25, 34, 64, 90]

Part B (15 points)

What is the time complexity of Bubble Sort in the best, average, and worst cases?

Solution: - **Best Case:** $O(n)$ - when the array is already sorted, we only need one pass to confirm no swaps are needed - **Average Case:** $O(n^2)$ - on average, we need to perform about half of all possible comparisons - **Worst Case:** $O(n^2)$ - when the array is reverse sorted, we need $n - 1$ passes, each making up to $n - 1$ comparisons

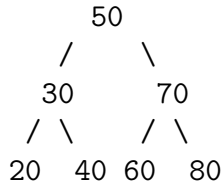
Problem 3: Binary Search Tree (25 points)

Question: Given the following sequence of insertions into an initially empty BST: [50, 30, 70, 20, 40, 60, 80]

Part A (15 points)

Draw the resulting BST after all insertions.

Solution:



Part B (10 points)

What are the inorder, preorder, and postorder traversals of this tree?

Solution: - **Inorder:** 20, 30, 40, 50, 60, 70, 80 - **Preorder:** 50, 30, 20, 40, 70, 60, 80 - **Postorder:** 20, 40, 30, 60, 80, 70, 50

Problem 4: Hash Tables (20 points)

Question: Consider a hash table with 7 slots using the hash function $h(k) = k \bmod 7$. Insert the following keys in order: [10, 22, 31, 4, 15, 28, 17] using linear probing to resolve collisions.

Solution:

Step-by-step insertion:

1. $h(10) = 10 \bmod 7 = 3 \rightarrow$ Insert 10 at index 3
2. $h(22) = 22 \bmod 7 = 1 \rightarrow$ Insert 22 at index 1
3. $h(31) = 31 \bmod 7 = 3 \rightarrow$ Collision! Linear probe: index 4 is empty $\rightarrow$ Insert 31 at index 4
4. $h(4) = 4 \bmod 7 = 4 \rightarrow$ Collision! Linear probe: index 5 is empty $\rightarrow$ Insert 4 at index 5
5. $h(15) = 15 \bmod 7 = 1 \rightarrow$ Collision! Linear probe: index 2 is empty $\rightarrow$ Insert 15 at index 2
6. $h(28) = 28 \bmod 7 = 0 \rightarrow$ Insert 28 at index 0
7. $h(17) = 17 \bmod 7 = 3 \rightarrow$ Collision! Linear probe: indices 4,5 occupied, index 6 is empty $\rightarrow$ Insert 17 at index 6

Final hash table:

Index:	0	1	2	3	4	5	6
Value:	28	22	15	10	31	4	17

Load Factor: $\alpha = \frac{7}{7} = 1.0$ (100% full)